

Classifications in Neural Networks

Maths for Machine Learning Project – LESTERLIN Raphaël,
DELANNOY Arthur, DONNAT Arthur and JABUA Indiko – Group 1

Table des matières

MNIST Classification	2
The Linear Model	2
Multi-Layer with Softmax output	2
Progression through epochs	3
The problematic cases	3
Tuning the epochs number	4
At 1 : one-off training	4
At 20 : Reaching the bottom of the plateau ?	4
Conclusions	4
Addendum : Layer effect study	5
CIFAR-10 Classification	5
Preliminary tests	5
Convolutional Neural Network	6
Conclusions	7
CBIS-DDSM Classification	7
Resources used	8
Necessary appendices	8
MNIST	9
CIFAR	16
CBIS-DDSM	22

MNIST Classification

First and foremost, we must thank Avishnish on Kaggle for providing the comprehensive MNIST dataset on the Kaggle platform. The link to it, with the links to other datasets and resources, will be put in the last section of this report.

The Linear Model

This basic model, the easiest to implement, has led to the discovery that the cross entropy preset function disposes of an integrated softmax function. We have thus not directly integrated to the definition of our model the softmax function to prevent the creation of a counter productive redundancy. We feel the need to add that we attempted a manual implementation of the gradient descent of a neural network, spending hours of our time to find that we made an extremely long sequential program with an exponential time complexity, taking at least a dozen of hours to fully complete (See Fig 0).

For faster code and better optimization, we then went on the use of Pytorch's integrated optimizer, with the CrossLogEntropy function as a criterion. We also decided to train the model through 5 epochs of training, since the time and cost of such process was reduced drastically, and could improve a lot the quality of the models. We also decided to split the dataset into a 60,000/10,000 split. The massiveness of the dataset gives us the luxury of providing massive amounts of training data to our model, as well as a statistically significant testing sample. Of course, the dataset is shuffled beforehand to prevent obvious biases to the model. After the training in a few seconds, we obtain Figure 1. We already get with this model an 8.7% error rate, quite impressive for such a simple model! We notice that the most misclassified numbers are twos, threes, fives, eights and nines, a distribution we will be confronting to the other models' realizations.

Multi-Layer with Softmax output

For the hidden layers of each neural network, we decided to provide each layer with exactly one hundred neurons. The reasoning behind was that using a multiple number of the number of neurons of the output layer would allow the system to fine-grain its analysis, without putting too much of a burden or create an hyperdimensionality risk. The ReLU activation function was used for all layers, since according to the course, it was the most popular activation function in machine learning. Maintaining it everywhere except the output layer permits us to keep coherence, as we will confront the performance of these models against each other. We used the acronyms MLS1 to refer to the model with one hidden layer, and MLS2 for the model with two hidden layers. After 5 epochs of training, we obtained figure 2 and 3.

We first notice that MLS1 dropped the error rate to 5%, a decrease of 3.7% compared to its predecessor. In these terms, MLS2, however, doesn't do that much of an improvement, with a 4.1% error rate, a 0.9% decrease compared to MLS1. Despite this,

MLS1 and MLS2 exhibit different distributions for misclassification: twos and nines hold the highest amounts in MLS1, with the number of two misclassified cratering in MLS2. Misclassified eights remain constant, showing a difficulty to improve accuracy across the two multi-layer systems. Nines still get a little bit more accurately classified in MLS2 but remain the champion of wrong predictions. We can also see that ones are the best classified only in MLS1 with a short lead, being overtaken by zeroes in the Linear and MLS2 systems. We can also observe that the models with the most layers also “cracked the code” quickly for fives.

Progression through epochs

Since we decided to train our models through multiple epochs (i.e., redoing the gradient descent calculation on the newly updated weights, through the whole training dataset again), an interesting aspect is to look at how our models “progress” through their training in figure 4.

The most striking thing is that the essential of the progress happens between the first and second epoch, where the curve has the steepest slope. The curve then slowly reaches a plateau with the more epochs they go through. Even if the multi-layer systems start with a loss superior to those of the Linear System in the first place, MLS1 and MLS2 both reach easily lower levels of loss compared to their predecessors.

Onto the set of incorrect predictions, we notice that, compared to the ratio of the sizes of the incorrect predictions set of the multi-layer models respectively and the Linear System, the proportions of similar incorrect predictions aren’t that far from the first proportion. In other words, all models are failing mostly in the same cases, with the multi-layered model grinding down bit by bit the remaining ambiguities.

The problematic cases

Decided to see what digits were troubling our models, we decided to pick 25 random inaccurate predictions and to display them via matplotlib. The limitations on the graphical capabilities of the library preventing us from seeing more, we conducted the operation on each models’ wrong prediction set to get better insight. See figures 5-7.

At first glance, we can clearly understand why the model had a hard time classifying some of these digits, since even with a human eye we couldn’t decipher quickly what digit this is (Image 464 in LS set, 219 in MLS1 set, 205 and 139 in MLS1). Models can also be surprised by unconventional writing, such as in image 34 and 793 of LS, or 339 in MLS2. We can see that across all three models, we can easily find true Nines picked, letting us witness the difficulty for our Machine Learning initiative to differentiate the number from others. The remaining patterns are either digits that models struggle the most on following their misclassification distributions, or just statistical noise following the low error rate of the model.

Tuning the epochs number

For pure experimentation reasons, what if we played with the number of training epochs ?

At 1 : one-off training

Let's leave only one backward pass to train our model. We see in figure 8 to 11 that with only one epoch of training, the error rate climbs to 10,6% for the Linear Model, MLS2 only faring better by 1,4%. Eights and fives are way more represented in the misclassified sets, before being taken over by twos and threes in MLS2. Surprisingly, despite the higher total loss, MLS2 still performs better than the linear system. Another remarkable fact is that both multi-layer systems are more prone to do wrong classifications dissimilar to those from the Linear system in this case, with a much higher wrong predictions set compared to the Linear system's one in this configuration. Distribution-wise, the twos, threes, eights and nine remain as in the original Linear system attempt, all equal troubles for the models.

At 20 : Reaching the bottom of the plateau ?

To see how better our models can get, let's quadruple the amount of epochs. Already at the loss level, we can see slow but steady progress in figure 12. All curves seem to follow a negative exponential curve, whose slope diminishes at the number of epochs grow. The Linear System model flattens the fastest, and the multi-layer systems outperforms it to the point that the size of the incorrect prediction set is less than 40% the size of the Linear system one!

This trend is demonstrated in the following errors rates (figure 13-15), with the Linear System barely outperforming its 5-epochs trained equivalent by 0.8%. MLS1 and MLS2 error rates, on the other hand, have an error rate almost cut in half! And the loss curve still isn't completely flattened. Distribution-wise, it is the threes and nines that causes trouble the most to those very trained models, the performance with the eights still getting better but hitting a ceiling too. Surprisingly, the ones are taking the place of most correctly classified in this category.

As we can see, only the most difform and difficult to instantly decipher graphically digits remain in the wrong predictions.

Conclusions

The study is clear: Multi-Layer Neural Networks in image classification outperforms in all cases simple Linear Systems. That performance gap widens even more as the models are trained through more epochs, as the multi-layer systems are able to "learn more" with the depth gained by several layers of neurons giving more margin to improve. The number of parameters, here the weights and biases of neurons, are key for a model to be able to grasp the full complexity of an image, and to detect

patterns from here. Even though it is a variable to use with restraint, as an increasing number of parameters might create overfitting or an unmanageable amount of noise. Models are also faced with a learning wall they can't fully get over: the amount of loss eliminated at each round of gradient descent diminishes until each step becomes insignificant, and Linear Models become stuck the fastest. Even if we can increase somewhat the margin of progress, the MNIST dataset is only a demo case study compared to the real complexity of image processing. Real images consist of squares of thousands of pixels over thousands of pixels long, with several color dimensions making their analysis by a machine much more complex. MNIST is only made of 28x28 pixel squares in greyscale, measuring intensities of black and white, and is not really representative of modern photography. As such, we aim further in this project to tackle the challenges of processing real images for concrete purposes.

Addendum : Layer effect study

Following the request of the professor, we tried to study the effect of adding more and more hidden layers onto the models, using similar sizes from our previous multi-layer models. After trying 3, 4, 5 and then 10 hidden layers, we have found that the 10 layer models had a loss at start way superior to any other model, and barely improved over 5 epochs. The other models didn't fare that much better! The more layers they got, the worse was the loss starting point, and at the end of the 5 epochs, they plateau'ed around a total loss of 50. A small improvement over the MLS2 model stopping at 63. Looking at the relationship to the Linear System, MLS10 has quite predictably an array of predictions error whose size is literally off chart relative to the LS. They all seem to be quite unique errors too. But the MLS3,4,5 also have their peculiarities: MLS4 has an unexplainable error similarity rate superior to its size comparative to LS' wrong predictions array, and MLS5 sees a first increase in wrongful predictions array size among the MLS variety. This lets us have a peek at the disadvantages of oversized neural networks, as the surpluses of neurons ends up being counterproductive. After deeper study, we see, in a new addendum, that the performance on the training dataset is similar, although less performant for both MLS2 and MLS5 compared to MLS10, quite surprisingly. We can thus affirm that this is not due to overfitting, but rather an increasing amount of noise created by the layers.

CIFAR-10 Classification

Preliminary tests

We retook our model design from the section above and decided to run it through five epochs of training based on the recommendation filed in the assignment document. In figures 17-20, the error rate is so great that we will speak in terms of its complementary metric: the accuracy. In a surprising turn of events compared to our first

section, the multi-layer models perform worse than their linear counterparts. The multi-layer model in grayscale only has an accuracy of 25.00% compared to the 27.25% of its linear counterparts, and in the full color channel exploitation models, the multi-layer has a 35.38% accuracy compared to 37.97% for its linear counterpart. This is pale in comparison to the accuracy reached by the presented articles listed, where the worst performing comparison had a 78,9% accuracy (21,1% error rate). This might be due to the presence of several layers overinterpreting a raw flattened vector, thus presenting the need of more complex processes to be able to get more satisfying results.

Convolutud Neural Network

To enhance the process, we used the library torchvision to correctly process our image dataset and be able to load it smoothly inside our network. Furthermore, we integrated a new type of layer, a Sequential to group more efficiently the Convolutional and Max-Pooling layers into several blocks, with a ReLU smoothening between the different layers. The first block is formed by a 3 to 64 2d Convolution layer, followed by a 64 to 64 one, both padding 1, then a Max-Pooling 2d. The second block consists of a Conv2d, still 64 to 64, followed by another 2d Max-Pooling. Finally, the last block flattens the result, and a Linear model processes the 4096 features into ten output neurons to get a class. The Stochastic Gradient Descent was picked as our optimizer. We obtained, as seen in figure 21, a 36.81% accuracy on the test set, below the expected potential. This might be, because five epochs with a learning rate of 0.01 might not be enough for the optimizer to converge fully to the best performance point, added to the fact that we did not use 3d filters as suggested. This is because we thought 2D filters might be better at processing 2D images; the results seems to contradict that hypothesis.

After some trial and error, we have decided to change the learning rate to 0.001 and pick the Adam optimizer, which yielded way more satisfying results. With the allowed freedoms of the second deposit, we shall present the improvements in this table:

Model	New Accuracy	Improvement
Gray Linear	26.46 %	- 0.79 %
Color Linear	36.08 %	- 1.89 %
Gray MLP	35.78 %	+ 10.78 %
Color Linear	45.89 %	+ 10.51 %

2D CNN	67.85 %	+ 31.04 %
3D CNN	70.13 %	+ 32.97 %

Conclusions

Despite our first errands, we have seen with the use of Adam optimizer the spectacular performance augmentation revealing the true power of Convolutional Neural Networks in image processing and classifying. A 30% increase in accuracy, only with changing the optimizer ! However, we notice that this change creates a slight decrease in accuracy for the linear models only, suggesting that this optimizer is more fit for multi-layer neural networks. The integration of the color factor into the training of the model leads to a significant improvement mostly in the 4 first models, while the switch from 2D based filters to 3D filters in the CNN only adds a little 3 % of accuracy. At the end of the day, we were able with a 3D CNN to be only 9% off the error rate of the models in the article “Convolutional Deep Belief Networks on CIFAR-10” (knowing that with a 70% accuracy of the model, we have a 30% error rate). Quite a performance! Figure 29 contains an additional chart summing up the performances of our models with the Adam configuration.

CBIS-DDSM Classification

With all that we have learned through the two previous sections, we have decided to apply another 2D CNN to the given breast cancer dataset. We first pre-processed the dataset by using a script to create a new path to copy into a folder tied to their true label, each image based on the CSV grid. Once we get these paths, we shuffle the list, split into test and training set using array splitting, we convert into grayscale using the Pillow library and resize to our 128 times 128 format, finally saving it into the appropriate folder with a file name stating whether they are part of the training or testing set, and their true label. The main script then retrieves the files, reapplying some transforms directly with torchvision’s transforms, before loading it all into data loaders and transfer it to the MedicalCNN model. It applies 3 successives convolutions followed immediately by a MaxPooling with a kernel on size 2: it passes from 1 in channel to 32 outs, 32 ins to 64 outs, and 64 ins to 128 outs respectively. The result is flattened, passed into a Linear Layer turning the 128x128 dimension into a single 256 element long line, filtered through ReLU and a 0.5 threshold, before passing through another Linear Layer showcasing the probabilities of belonging either to the malign or benign class. To better the training, we decided to iterate it through 10 epochs.

Thus, with the binary classification format being perfect for that, we set up a confusion matrix that is available in Figures 30-31. We consider the Malignant Class to be a positive (detection of a cancer), and the Benign to be the complementary negative. As a recall, one positive or negative is said to be true if the prediction matches the true label, false if it is the opposite case.

Classification Results	Negatives	Positives
True	38.26 %	28.03 %
False	16.29 %	17.42 %

It must be noted that our testing sample only consists of 264 elements, a smaller size compared to our previous sets. But the most important information out of this table is the 16.29% of false negatives, corresponding to 16% of malign tumor incorrectly classified as benign. If a misclassified benign case can just lead to a stress spike, a misclassification of a malign tumor implies that the doctor and patient can leave a cancer potentially proceed to metastasis, developing freely without the scrutiny of professionals. And in that phase, it can often be fatal. As such, the application of AI in the medical sector necessitates particularly strong reviews by healthcare and medical staff, including double checking, to make sure that diseases and afflictions encountered by the human body doesn't add to their death counts. This last project model is of course not fit at all for real medical purposes with such a high rate and would necessitate magnitudes of bigger datasets to the very least to refine sufficiently the parameters.

Resources used

Datasets :

- [MNIST Original](#)
- [CBIS-DDSM: Breast Cancer Image Dataset](#)
- [CIFAR-10 - v2 2024-05-30 3:48pm](#)

Github : <https://github.com/Miner205/Mathematics-for-Machine-Learning-Project>

Necessary appendices

```
"""entree = torch.tensor(mnist_data[0], dtype=torch.float32)
exit = LS(entree)
print("exit :", exit.tolist())
print(mnist_label[0]) # Initial run with all weights to 1

learning_rate = 0.1 # base learning rate for the rest of the program

for k in range(0,5): # To prevent overfitting, equal amount of gradient descent iteration over the whole training set (5 iterations
    for l in range(0,len(mnist_data)-10000): # Over each image of the training dataset
        true_label = [0 for k in range(0,10)] # Updates at each new iteration
        true_label[int(mnist_label[l])] = 1 # For log loss, converts scalar true label into signal with 0 or 1 if said image belong

        with torch.no_grad(): # Mandatory else it doesn't let me apply manually the update
            for i in range(0,len(working_weights_mn)): # for each neuron
                for j in range(0,len(working_weights_mn[0])): # for each among the 784 weights
                    working_weights_mn[i][j] = working_weights_mn[i][j] - np.float64(learning_rate*1/784*(exit[i] - true_label[i]))*

        # new forward pass
        LS.layer.weight.data = torch.tensor(np.array(working_weights_mn, dtype=np.float32))
        entree = torch.tensor(mnist_data[l], dtype=torch.float32)
        exit = LS(entree) # removed prints for pure calculation, only prints when reaching threshold
        if l % 6000 == 0: # Set at 10% bar
            print("Model training at epoch " + str(k) + " trained at " + str((l / 6000)*10) + "%")

predicted = []
for i in range(60000, len(mnist_data)):
    entree = torch.tensor(mnist_data[i], dtype=torch.float32)
    exit = LS(entree)
    max_i = 0
    for j in range(0,len(exit)): # Looks for index of max probabilities in output array
        if exit[j] > exit[max_i]:
            max_i = j
    predicted.append(float(max_i))""" # Other legacy manual code, for demo purposes
```

Fig. 0 : The manual weight update gradient descent. Way too unoptimized.

MNIST

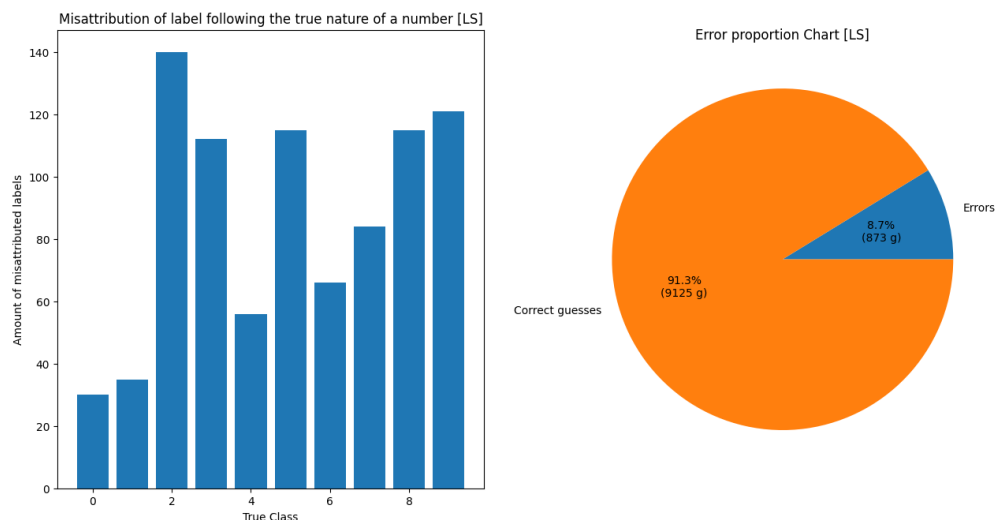


Fig. 1 : Distribution of incorrect predictions and error rate visualization for the Linear System.

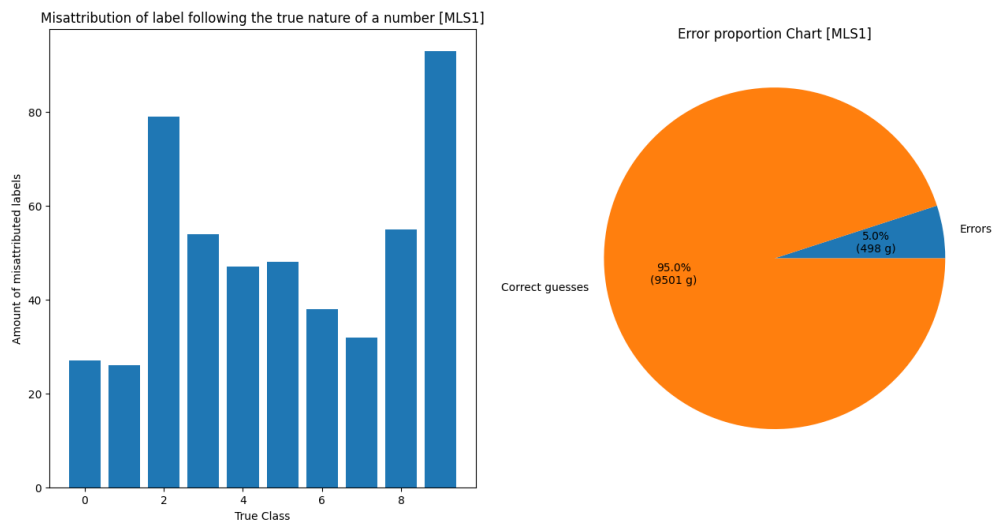


Fig. 2 : Distribution of incorrect predictions and error rate visualization for MLS1.

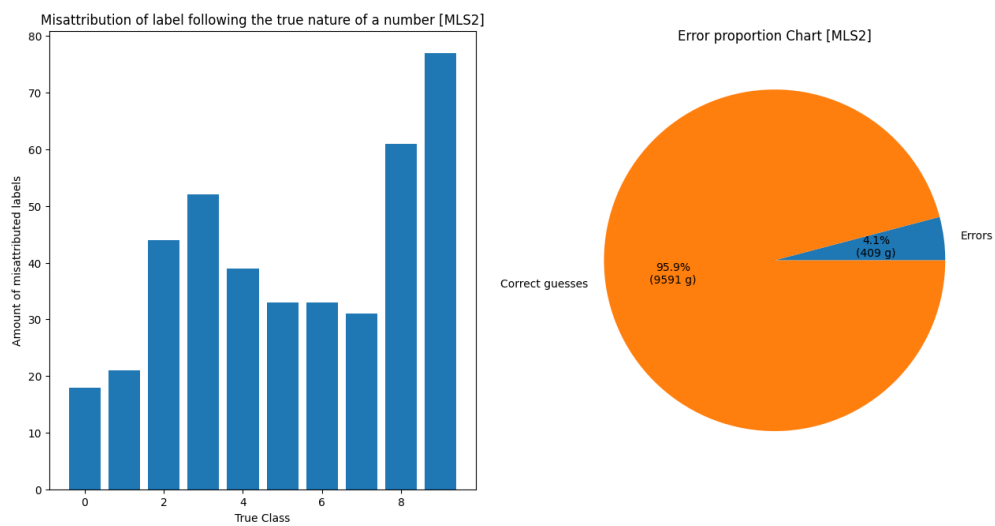


Fig. 3 : Distribution of incorrect predictions and error rate visualization for MLS2.

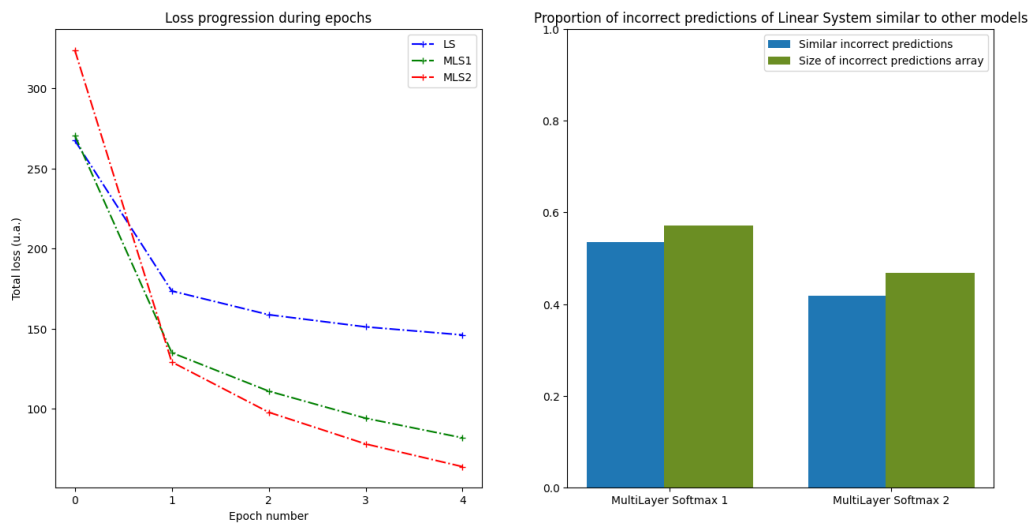


Fig. 4 : Total Log loss according to epoch and similarity rate across wrong prediction set.



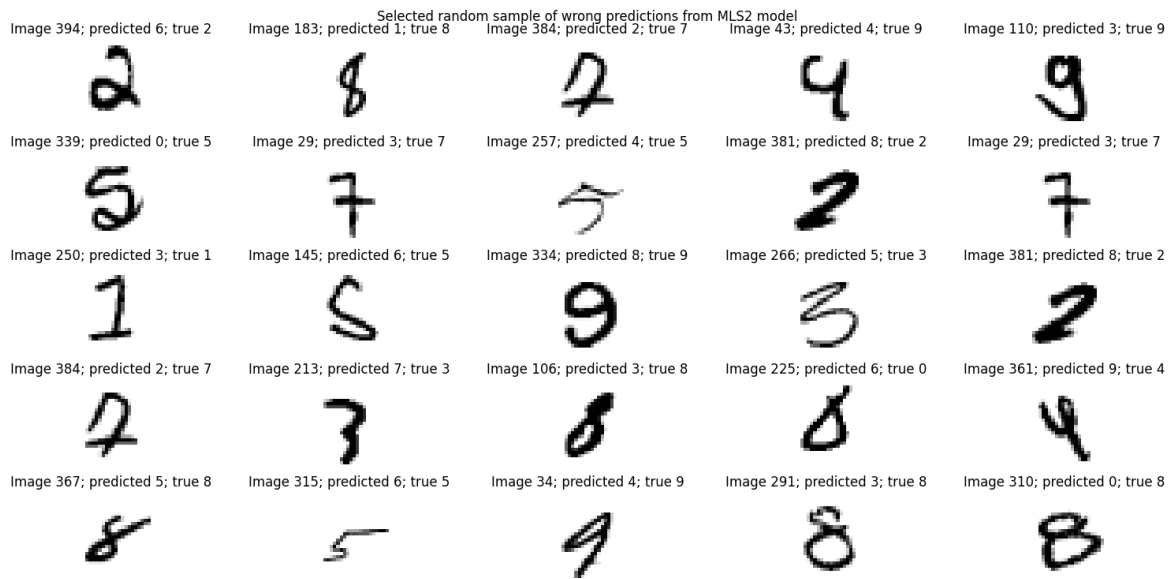
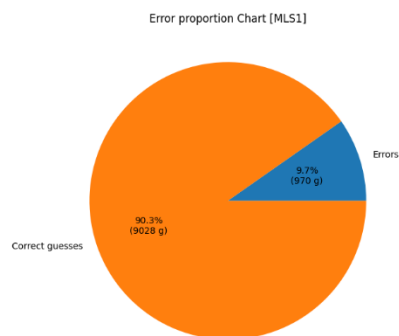
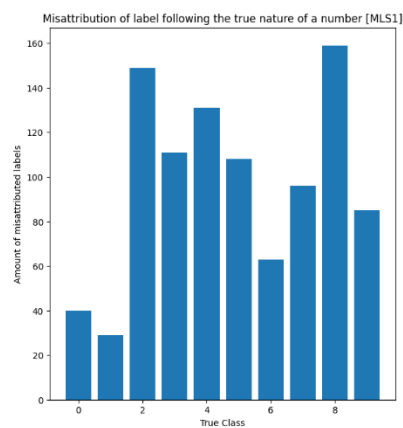
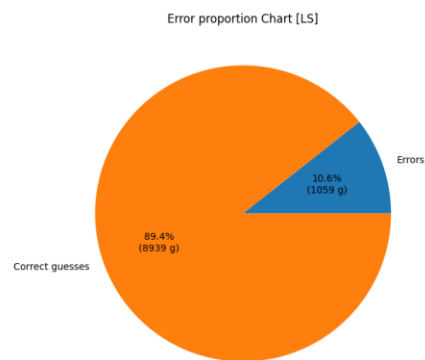
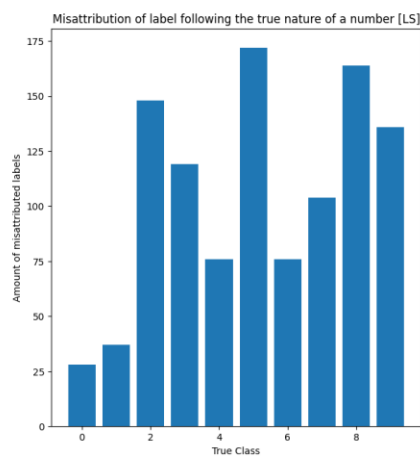


Fig. 5-7 : Sample of incorrect predictions from each model, with predicted and true label.



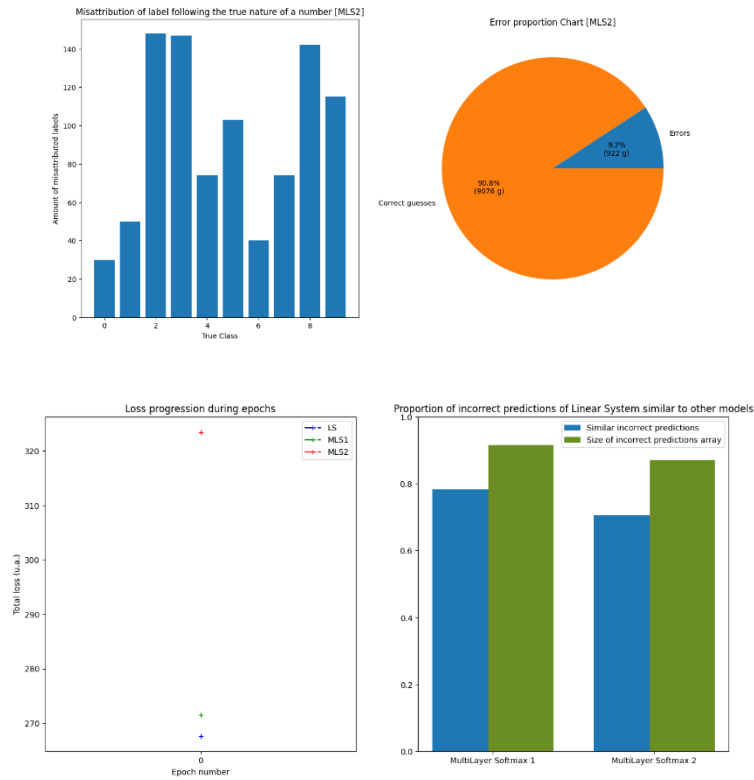


Fig. 8-11 : Similar graphs for all three models as above, for only 1 epoch of training.

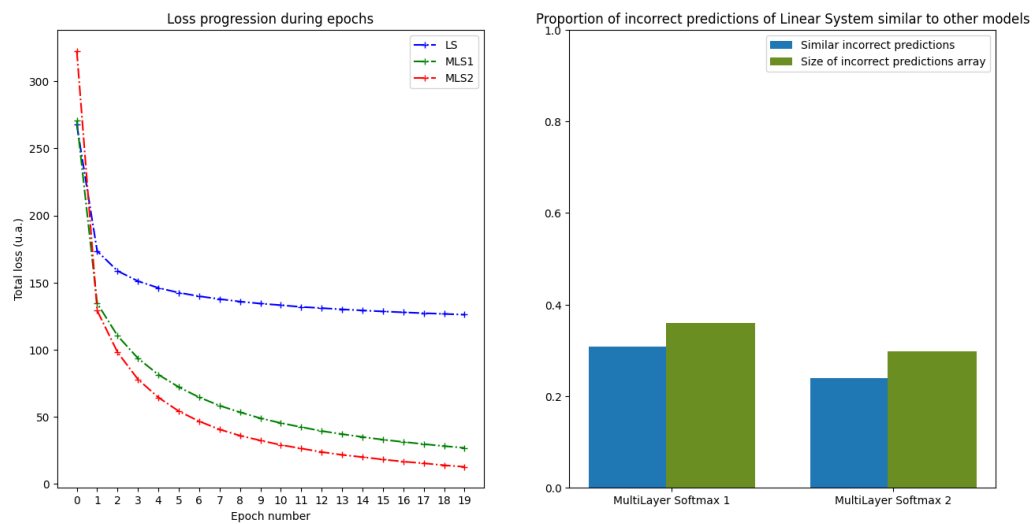


Fig. 12 : Loss progression and incorrect predictions set similarity at 20 epochs.

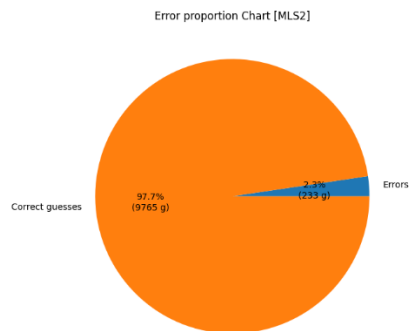
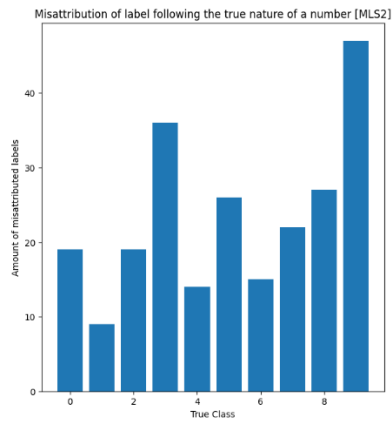
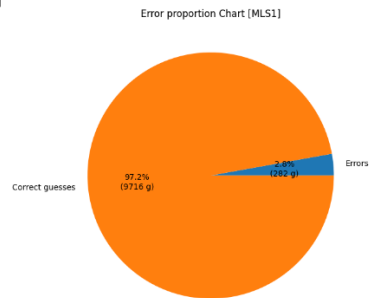
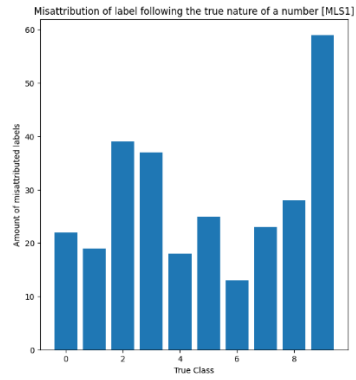
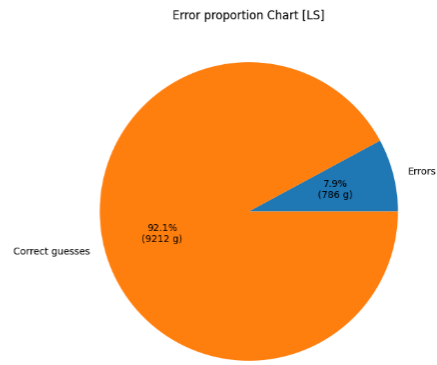
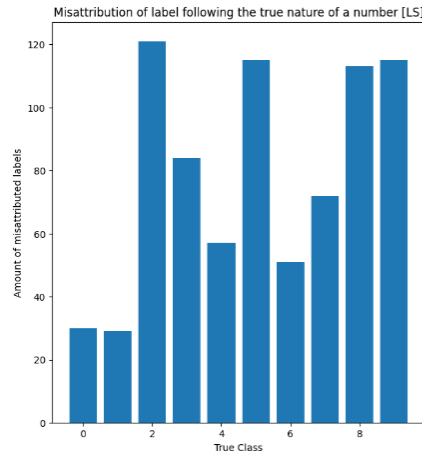


Fig. 13-15 : Error rates and misattribution distribution for all models with 20 epochs.

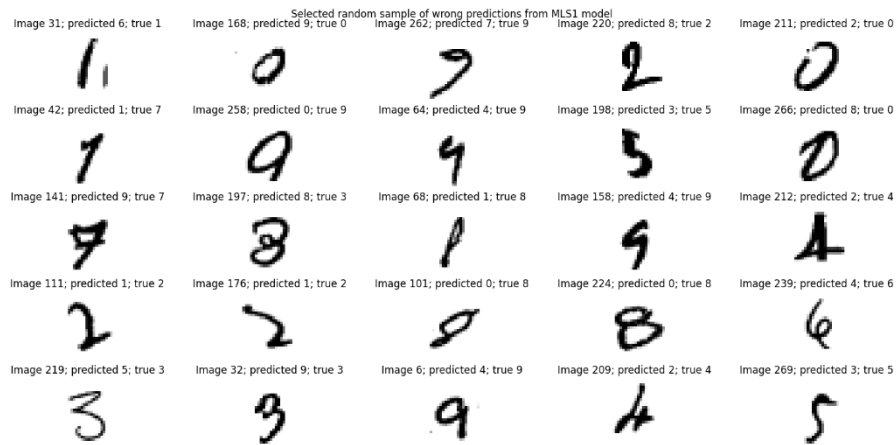


Fig. 16 : Remaining incorrect predictions as shown by matplotlib.

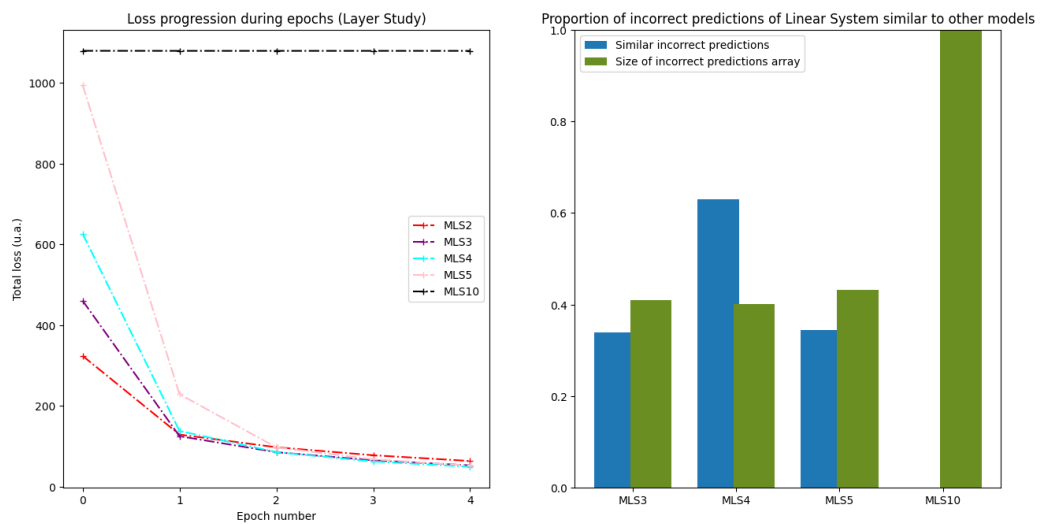


Fig. A1 : Charts of loss and incorrect predictions study for the MLS3, 4, 5 and 10 models.



Fig. A2 : Error proportion over the training dataset for MLS10, 2 and 5.

CIFAR

```
--- Testing Linear Grayscale ---

--- Training Linear_CIFAR_Gray ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 716.9981
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 694.0894
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 685.9454
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 681.6167
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 678.5447

--- Training of Linear_CIFAR_Gray Completed ---
=> Verification: Accuracy on Test Images: 27.25%
```

```
--- Testing Linear Color ---

--- Training Linear_CIFAR_Color ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 661.5721
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 618.9349
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 606.5603
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 599.6763
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 594.7761

--- Training of Linear_CIFAR_Color Completed ---
=> Verification: Accuracy on Test Images: 37.97%
```

```
--- Testing MLP Grayscale ---

--- Training MLP_CIFAR_Gray ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 749.1709
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 728.1021
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 710.0778
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 699.1144
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 689.9894

--- Training of MLP_CIFAR_Gray Completed ---
=> Verification: Accuracy on Test Images: 25.00%
```

```
--- Testing MLP Color ---

--- Training MLP_CIFAR_Color ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 730.6350
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 672.2972
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 640.3739
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 622.1553
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 609.4059

--- Training of MLP_CIFAR_Color Completed ---
=> Verification: Accuracy on Test Images: 35.38%
```

Fig. 17-20 : Screenshots of runs of Linear and MLP models (using Optimizer SGD and $lr=0.01$).

```

--- Testing CNN ---

--- Training CIFAR_CNN ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 756.7105
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 741.8249
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 677.5670
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 644.4741
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 606.9674

--- Training of CIFAR_CNN Completed ---
=> Verification: Accuracy on Test Images: 36.81%

```

Fig. 21 : Runs of the CNN 2D CIFAR model (using Optimizer SGD and $lr=0.01$).

```

--- Testing CNN 3D ---

--- Training CIFAR_CNN_3D ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 755.6430
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 721.9670
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 661.2430
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 624.4516
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 592.7324

--- Training of CIFAR_CNN_3D Completed ---
=> Verification: Accuracy on Test Images: 37.16%

```

Fig. 22 : Runs of the CNN 3D CIFAR model (using Optimizer SGD and $lr=0.01$).

```

--- Training Linear_CIFAR_Gray ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 697.6923
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 680.5328
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 676.8044
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 676.0565
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 674.1392

--- Testing Linear_CIFAR_Gray ---

--- Training and Testing of Linear_CIFAR_Gray Completed ---
=> Verification: Accuracy on Test Images: 26.46%

```

Fig. 23 : Runs of the Linear Gray model with Adam optimizer and learning rate to 0.001.

```

--- Training Linear_CIFAR_Color ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 638.8284
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 602.4610
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 598.7772
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 588.8070
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 587.5452

--- Testing Linear_CIFAR_Color ---

--- Training and Testing of Linear_CIFAR_Color Completed ---
=> Verification: Accuracy on Test Images: 36.08%

```

Fig. 24 : Linear Color with the modified optimizer (Adam) and learning rate to 0.001.

```

--- Testing MLP Grayscale ---

--- Training MLP_CIFAR_Gray ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 681.3031
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 634.0285
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 611.5338
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 595.8299
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 584.8619

--- Testing MLP_CIFAR_Gray ---

--- Training and Testing of MLP_CIFAR_Gray Completed ---
=> Verification: Accuracy on Test Images: 35.78%

```

Fig. 25 : Multi Layer model in grayscale with the same new conditions.

```

--- Training MLP_CIFAR_Color ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 621.5327
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 557.2205
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 530.9465
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 507.1887
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 497.5496

--- Testing MLP_CIFAR_Color ---

--- Training and Testing of MLP_CIFAR_Color Completed ---
=> Verification: Accuracy on Test Images: 45.89%

```

Fig. 26 : Multi Layer model in color with the same conditions.

```

--- Training CIFAR_CNN ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 531.8520
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 396.4224
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 333.1507
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 288.8224
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 260.3070

--- Testing CIFAR_CNN ---

--- Training and Testing of CIFAR_CNN Completed ---
=> Verification: Accuracy on Test Images: 67.85%

```

Fig. 27 : CNN in 2d with the same conditions, and quite an improvement.

```

--- Training CIFAR_CNN_3D ---
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 1/5 - Loss: 542.0796
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 2/5 - Loss: 406.0088
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 3/5 - Loss: 336.0328
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 4/5 - Loss: 291.3328
> Processed 100/329 batches...
> Processed 200/329 batches...
> Processed 300/329 batches...
Epoch 5/5 - Loss: 259.2173

--- Testing CIFAR_CNN_3D ---

--- Training and Testing of CIFAR_CNN_3D Completed ---
=> Verification: Accuracy on Test Images: 70.13%

```

Fig. 28 : CNN in 3d with the same conditions.

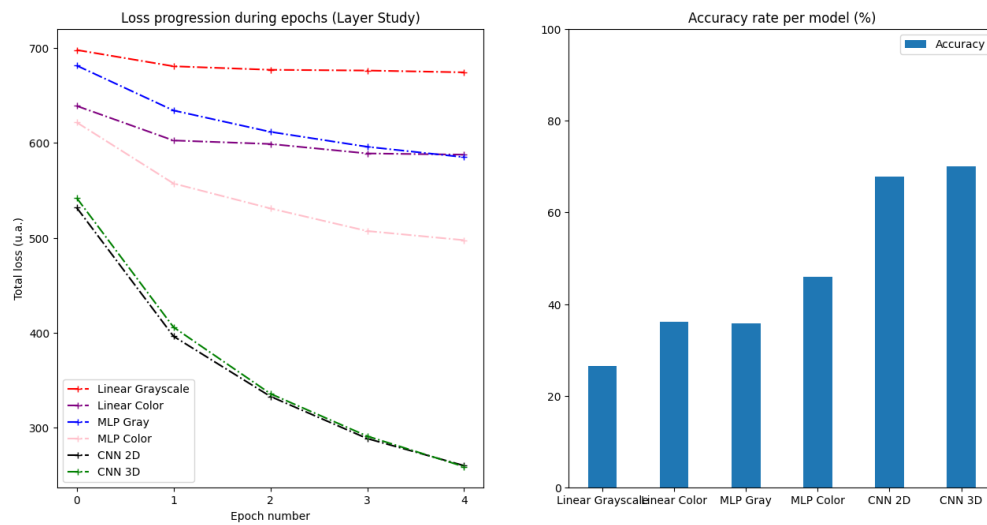


Fig. 29 : Chart Summaries of model performance. As we can see, the CNN models with the Adam optimizer have a way steeper learning curve, showing their leadership.

CBIS-DDSM

```

--- Training MedicalCNN ---
Epoch 1/10 - Loss = 23.1061
Epoch 2/10 - Loss = 22.1399
Epoch 3/10 - Loss = 21.5980
Epoch 4/10 - Loss = 20.9008
Epoch 5/10 - Loss = 19.5338
Epoch 6/10 - Loss = 18.5782
Epoch 7/10 - Loss = 18.1602
Epoch 8/10 - Loss = 15.9434
Epoch 9/10 - Loss = 14.6347
Epoch 10/10 - Loss = 13.0173

--- Testing MedicalCNN ---

--- Training and Testing of MedicalCNN Completed ---

Test Accuracy: 66.29%

Confusion Matrix:
[[101  46]
 [ 43  74]]

```

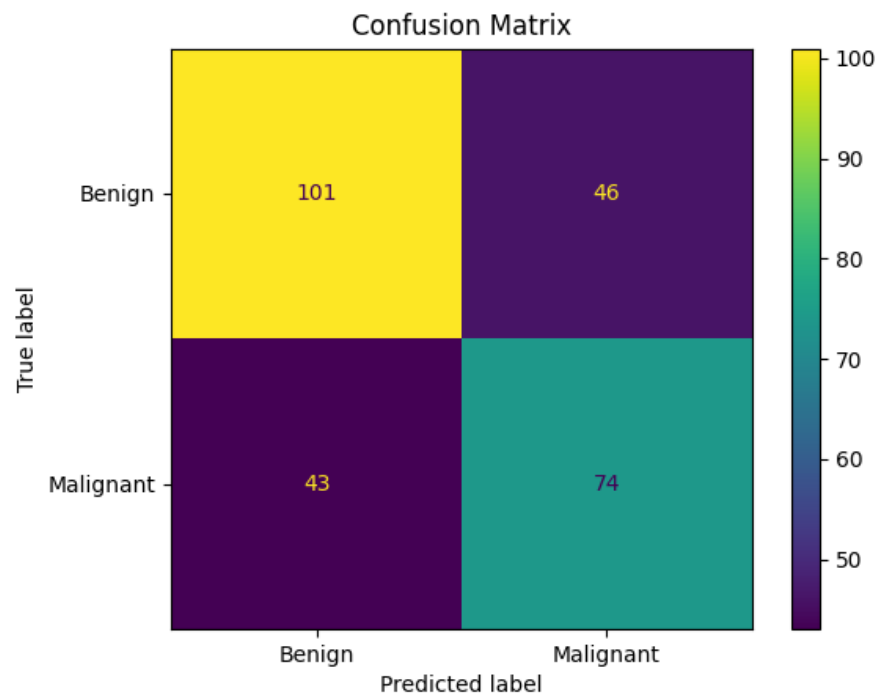


Fig. 30-31 : Runs of the CNN CBIS-DDSM model (using Optimizer Adam and $lr=0.001$).